AULA 41

Transactional Outbox, Inbox e idempotência

Java

Conceitos, exemplos em Java e desafio aplicado

29/10/2026 · 14:00–17:00 · Java e Quarkus

FIAP · FORMAÇÃO EM BACKEND JAVA · TURMA 2

Roteiro do encontro: 3 horas

- 14:00–14:05 Abertura, cenário e objetivos.
- 14:05–15:15 Teoria com exemplos e discussão.
- 15:15–15:45 Prática guiada pelo professor.
- 15:45–15:55 Intervalo.
- 15:55–16:45 Desafio.
- 16:45–17:00 Discussão, demonstrações e dúvidas.

JAVA

70 + 100

70 min de abertura e teoria. 100 min de prática e discussão, com 10 min de intervalo.

Objetivos de aprendizagem

- Compreender a falha entre gravar dados e publicar uma mensagem.
- Gravar negócio e outbox em uma transação local.
- Deduplicar o consumo com chave única e transação.
- Testar duplicatas, falhas do relay e retomada do processamento.

JAVA

Entrega

Ao final, cada dupla demonstra a solução e explica uma decisão técnica.

O cenário da aula

- Criar um pedido também precisa notificar o faturamento.
- Gravar no banco e publicar no broker são duas operações.
- Uma falha entre elas pode deixar um pedido sem evento.
- O laboratório persiste a intenção de publicar junto do pedido.

JAVA

Problema

A aplicação precisa sobreviver à falha entre os dois sistemas.

CONCEITOS E EXEMPLOS

Transactional Outbox, Inbox e idempotência

Uma decisão de arquitetura aplicada a um problema observável

A janela de falha da gravação dupla

- Banco primeiro: o processo cai antes de publicar → pedido sem evento.
- Broker primeiro: o banco rejeita depois → evento sem pedido (fantasma).
- Repetir a publicação para "garantir" → evento duplicado.
- Não existe uma transação que abranja banco e broker: isso exigiria que os dois falassem o mesmo protocolo de commit coordenado, e os brokers modernos não falam.
- Se o pedido A foi gravado antes do B, o evento de A precisa chegar antes do de B — e "publicar depois do commit" não garante isso.

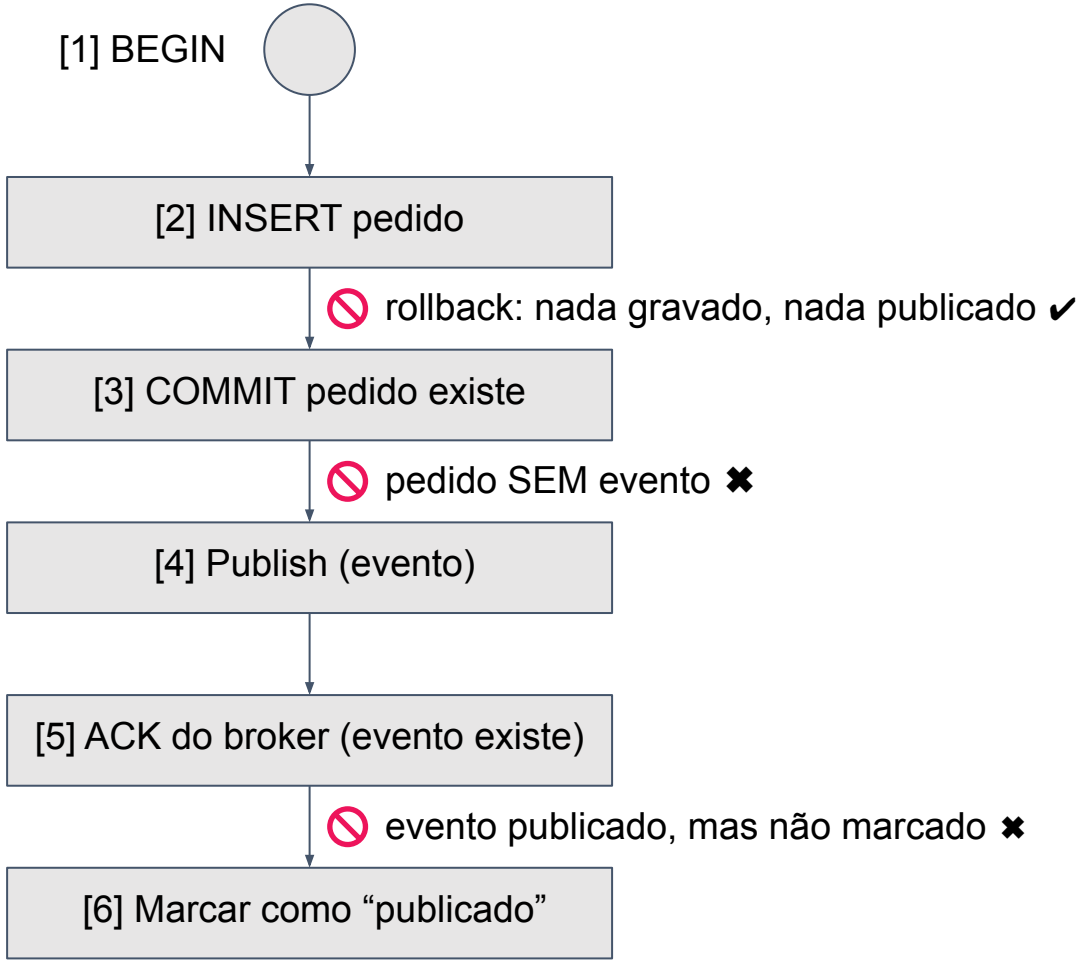
JAVA

Consequência

O projeto (design) deve considerar falhas e repetição.

A outbox resolve porque o relay lê a tabela na ordem em que foi gravada.

Possibilidades de falhas



JAVA

Possibilidades de contorno

Assumindo que retry seria uma necessidade, em qual das três quedas o retry resolve?

Só funciona com a outbox (sabe o que faltou) e com a inbox (tolera o que repetiu).

Transactional Outbox

- A transação grava o pedido e uma linha de outbox.
- O commit torna ambos visíveis ao mesmo tempo.
- Um relay lê a outbox e publica o evento.
- O relay descobre o que publicar de duas formas: consultando a tabela de tempos em tempos (polling, o que faremos hoje) ou lendo o log de transações do banco (log tailing).
    - Polling é um SELECT ... WHERE enviado = false a cada N segundos. Simples, funciona em qualquer banco, custa um pouco de latência e uma consulta repetida.
    - Log tailing é uma ferramenta que lê o diário interno do banco (no Postgres, o WAL) e enxerga cada INSERT na outbox no instante em que acontece: latência menor, zero consulta, mais uma peça de infraestrutura.

JAVA

Garantia local

O pedido e a intenção de publicar compartilham a mesma transação.

O padrão garante que o evento sai se, e somente se, o pedido foi gravado, e na ordem em que foi gravado, sem precisar de protocolo coordenado.

Publicação e duplicação

- O relay publica e aguarda a confirmação do broker.
- Depois, marca a linha como enviada.
- Se cair entre esses passos, poderá publicar novamente.
- Consumidores devem suportar o mesmo eventId repetido.
- No cenário da aula:
    - o relay lê o evento PedidoCriado 123, publica, o broker confirma — e o processo morre antes do UPDATE outbox SET enviado = true.
    - Quando sobe de novo, a linha ainda está enviado=false, então publica de novo.
    - O faturamento recebe PedidoCriado 123 duas vezes. Sem inbox, fatura duas vezes.

JAVA

At-least-once

A outbox não oferece processamento único de ponta a ponta sozinha.

Isso não é um defeito da outbox, pois brokers de mercado (Kafka, RabbitMQ, SQS) já entregam *at-least-once*; o consumidor precisa ser idempotente.

Inbox e consumo idempotente

- O consumidor identifica a mensagem por eventId estável.
- Uma tabela de mensagens processadas (a inbox) guarda o par (consumidor, eventId) com chave única.
- A mesma transação registra a chave e aplica o efeito de negócio.
- No PostgreSQL, INSERT … ON CONFLICT DO NOTHING devolve 0 linhas quando a chave já existe: duplicata detectada sem consulta prévia e sem exceção..
- A confirmação da mensagem ocorre após o commit.
- Há pelo menos duas formas de guardar o eventId:
    - tabela separada (nossa inbox de exemplo); ou
    - dentro da própria entidade de negócio (ex.: coluna ultimo_event_id), mas acopla controle com negócio.

JAVA

Atomicidade

Registrar a chave antes de uma falha só funciona com rollback conjunto.

Concorrência e chave única

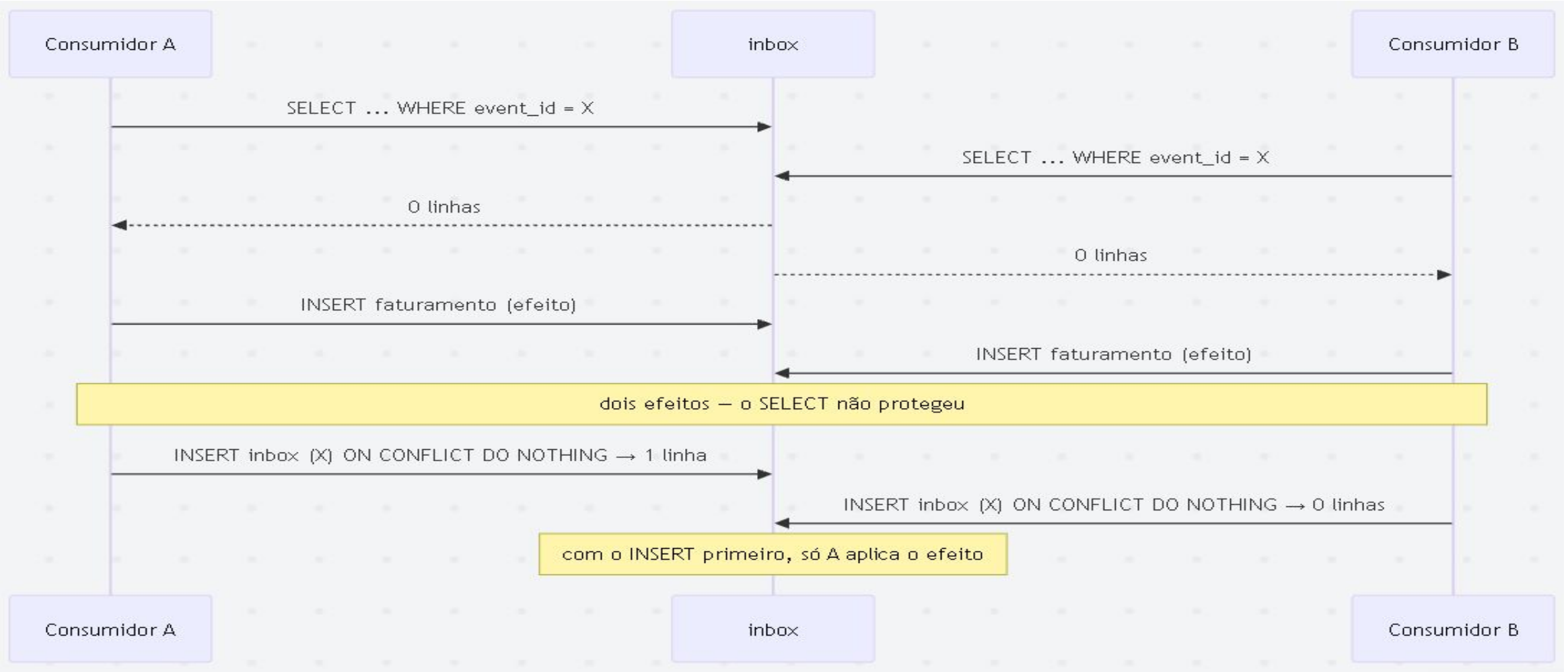
- Duas instâncias podem receber a mesma mensagem.
- Consultar antes de inserir não impede a corrida: duas instâncias podem consultar ao mesmo tempo e as duas veriam 'ainda não processado'.
- Só a chave única no banco decide: a segunda inserção é rejeitada (ou ignorada com DO NOTHING), não importa quem chegou antes (restrição única no banco serializa).

JAVA

Identidade

Um novo UUID a cada tentativa destrói a deduplicação. O UUID nasce em criar() e o relay apenas lê.

A corrida entre dois consumidores



Operação e ordenação

- Use agregado e versão quando a ordem por pedido importar.
- Um relay concorrente precisa de estratégia de claim e recuperação.
- Monitore idade da outbox, tentativas e erros persistentes.
- Defina retenção da inbox conforme a janela de replay.

JAVA

Limite

Deduplicação expirada pode permitir reaplicar um evento antigo.

Estrutura mínima no PostgreSQL

schema.sql

```
CREATE TABLE outbox (
  event_id   uuid PRIMARY KEY,
  pedido_id  bigint NOT NULL,
  tipo       text NOT NULL,        -- 'PedidoCriado'
  versao     int  NOT NULL,        -- 1
  payload    text NOT NULL,        -- JSON com pedidoId, clienteId, valor
  criado_em  timestamptz NOT NULL, -- ordem do relay
  enviado    boolean NOT NULL DEFAULT false,
  tentativas int NOT NULL DEFAULT 0);

CREATE TABLE inbox (
  consumidor  text NOT NULL,
  event_id    uuid NOT NULL,
  recebido_em timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (consumidor, event_id));

CREATE TABLE faturamento (          -- o "efeito" observável
  id bigserial PRIMARY KEY,
  pedido_id bigint,
  event_id uuid,
  valor numeric(12,2));
```

- As tabelas de negócio ficam no mesmo banco da transação local.
- O payload leva tudo o que o consumidor precisa (id, cliente, valor), para ele não ter de consultar o produtor.
- tipo e versao permitem evoluir o evento (PedidoCriado v1 → v2) sem quebrar consumidores; criado_em dá a ordem do relay.

Pedido e intenção no mesmo commit

Exemplo.java

```java
@Transactional
public void criar(Pedido pedido) {
  entityManager.persist(pedido);
  var evento = new OutboxEvent();
  evento.eventId = UUID.randomUUID();
  evento.pedidoId = pedido.id;
  evento.payload = serializar(pedido);
  entityManager.persist(evento);
}
```

- Pedido e OutboxEvent são entidades JPA.
- Uma exceção deve desfazer as duas inserções.
- O UUID pertence ao evento, não à tentativa de envio.

Consumidor com deduplicação atômica

Exemplo.java

```
@Transactional
public void consumir(UUID eventId, long pedidoId) {
  int novas = em.createNativeQuery("""
    INSERT INTO inbox(consumidor, event_id)
    VALUES ('faturamento', :id)
    ON CONFLICT DO NOTHING
    """).setParameter("id", eventId).executeUpdate();
  if (novas == 0) return;
  aplicarEfeito(pedidoId);
}
```

- em é o EntityManager injetado.
- O efeito usa o mesmo banco e transação.
- Se o efeito falhar, a inserção na inbox também reverte.

A fronteira exata de cada garantia

Garantia	Quem dá	Até onde vale
Pedido e evento existem juntos ou não existem	transação do produtor (criar)	até o commit no banco do produtor
O evento será publicado pelo menos uma vez	relay + outbox	até o ACK do broker (pode repetir)
O efeito acontece exatamente uma vez	transação do consumidor (inbox + efeito)	só para efeito no mesmo banco

JAVA

Cada linha é uma transação local diferente

Exactly-once de ponta a ponta não existe: existe at-least-once na entrega + idempotência no consumo.

Prática guiada: preparação

- Projeto com rest-jackson, hibernate-orm, jdbc-postgresql (+ smallrye-openapi para o Swagger). Com Docker no ar, o Dev Services sobe o PostgreSQL sozinho no ./mvnw quarkus:dev.
- Entidades Pedido e OutboxEvent; import.sql cria inbox e faturamento.
- Implemente criar e confira as contagens após um POST /pedidos.
- O broker é simulado: POST /simular/consumir/{eventId} entrega o mesmo evento ao consumidor quantas vezes quisermos.

JAVA

30 minutos

A simulação isola o padrão antes da integração com mensageria real.

Relay didático e confirmação

- Um único worker busca linhas com enviado=false.
- Ele publica usando o eventId persistido.
- Somente após a confirmação marca enviado=true.
- Simule a queda antes da marcação e execute o worker novamente: POST /relay/run?falharAntesDeMarcar=true derruba o worker entre publicar e marcar; rode de novo e veja a duplicata.

JAVA

Escopo

O relay didático usa um worker.

Concorrência fica para as reflexões finais ou extensões do desafio.

Roteiro da demonstração

- Crie um pedido e confira uma linha de outbox.
- Provoque uma falha antes do commit e confira rollback.
- Envie duas vezes o mesmo eventId ao consumidor.
- Verifique uma linha de inbox e um único efeito de negócio.

JAVA

Evidência

As contagens devem demonstrar o comportamento observado.

Desafio: entrega confiável

- Preencha tipo, versao e criado_em na outbox e ordene o relay por criado_em.
- Relay de worker único com reenvio após falha: enviado=true só depois da confirmação do broker.
- A tabela faturamento é o efeito observável: conte-a.
- Entregue a mesma mensagem cinco vezes preservando o eventId (/simular/consumir/{id}/vezes/5).

JAVA

50 minutos

A criação do pedido e o efeito do consumidor usam transações separadas.

Desafio: critérios de entrega

- Rollback da criação deixa zero pedido e zero outbox novos.
- Cinco entregas do mesmo evento geram um efeito de faturamento.
- Falha no efeito permite nova tentativa com o mesmo eventId.
- README inclui SQL de conferência e roteiro de falhas (cole a saída dos COUNT(*) e a coleção Postman verde).

JAVA

Aceite

A inbox não pode impedir a recuperação de uma tentativa que falhou.

Correção comentada

- A transação do produtor reúne pedido e outbox.
- A transação do consumidor reúne inbox e efeito.
- A chave única protege contra corrida entre instâncias.
- O relay ainda pode duplicar mensagens e precisa de consumidor idempotente.

JAVA

Revisão

Exatamente uma linha na inbox não prova um efeito externo único.

Reflexões

- E se dois consumidores receberem o mesmo evento ao mesmo tempo?
- E se dois relays lerem a outbox ao mesmo tempo — e um deles cair no meio?
- E se os eventos de um mesmo pedido chegarem fora de ordem?
- Por quanto tempo a inbox precisa lembrar de um evento?

JAVA

Para pensar

Lição de casa: escolha uma e tente quebrar o seu lab a partir do conteúdo de “aula41-solucao”.

Discussão e fechamento

- O relay publicou e caiu antes de marcar. O que vimos acontecer — e quem resolveu?
- Uma cobrança HTTP está protegida pela transação do banco?
- Se o legado fosse Oracle e o serviço novo PostgreSQL, onde ficaria a outbox?
- Documente a fronteira exata de cada garantia.

JAVA

Aprendizagem

Outbox preserva a intenção. Inbox coordena deduplicação e efeito local.

Referências e preparação

- https://microservices.io/patterns/data/transactional-outbox.html
- https://microservices.io/patterns/communication-style/idempotent-consumer.html
- https://quarkus.io/guides/transaction
- https://quarkus.io/guides/hibernate-orm
- https://quarkus.io/guides/databases-dev-services

JAVA

Professor

Prepare as dependências antes da aula e mantenha as versões usadas no README.